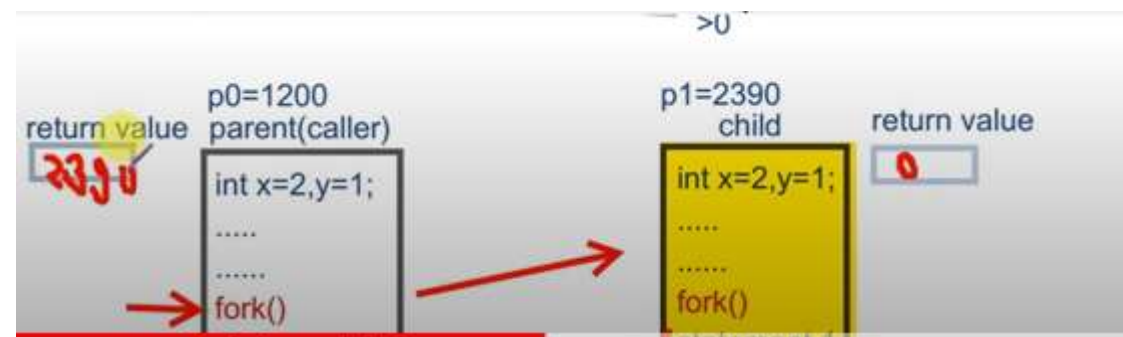
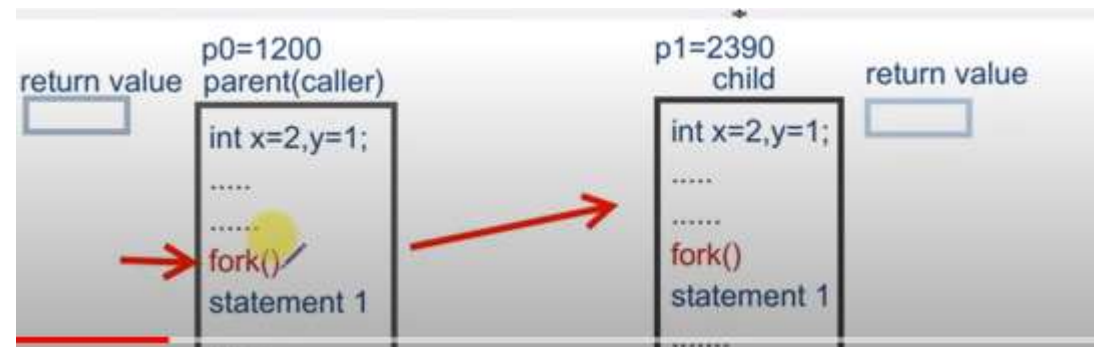


System call `fork()` is used to create processes. It takes **no arguments** and **returns a process ID**. The purpose of `fork()` is to create a new process, which becomes the child process of the caller. After a new child process is created, both processes will execute the next instruction following the `fork()` system call.

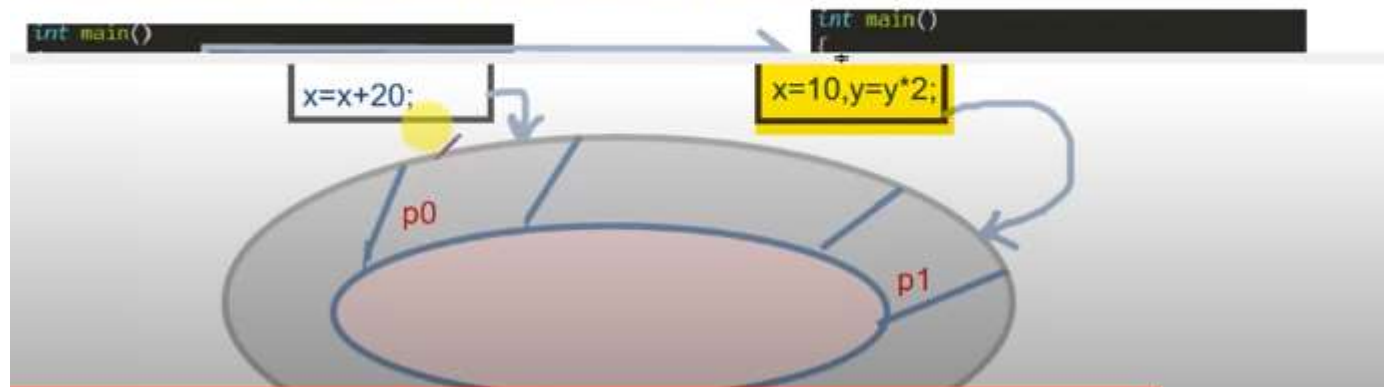
- To create child process we use `fork()`. `fork()` returns :
 - <0 fail to create child (new) process
 - $=0$ for child process
 - >0 i.e process ID of the child process to the parent process. When >0 parent







Since both processes have identical but separate address spaces, those variables initialized before the `fork()` call have the same values in both address spaces. Since every process has its own address space, any modifications will be independent of the others. In other words, if the parent changes the value of its variable, the modification will only affect the variable in the parent process's address space. Other address spaces created by `fork()` calls will not be affected even though they have identical variable names



```
#include<stdio.h>
```

```
int main()  
{  
    printf("hello\n");  
    fork();  
    printf("bye\n");  
}
```

C

1x
2X
3v

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
1 int pid=fork();
```

```
2 if(pid>0)
```

```
3     printf("parent ✓ pid_of_child=%d\n",pid); ✓
```

```
4 else if(pid==0)
```

```
5     printf("child return value=%d\n",pid);
```

```
6 else
```

```
7     printf("unseccessfull pid=%d\n",pid);
```

```
8     return 0;
```

```
9 }
```

parent process pid=1133

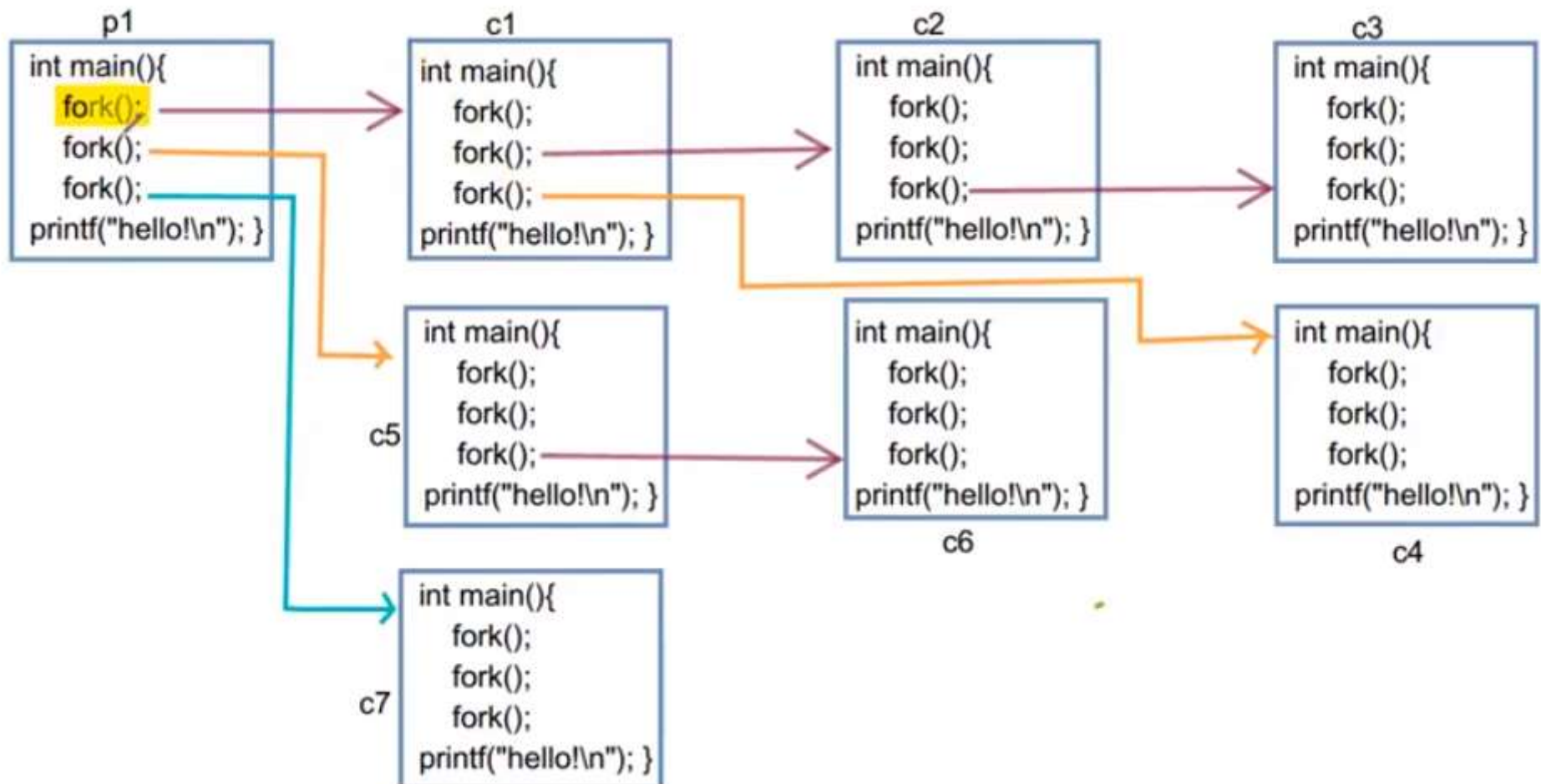
child process

pid=1120

pid = 1130

pid = 0

```
#include<stdio.h>
int main()
{
    int pid=fork();
    if(pid>0)
        printf("parent   pid_of_child=%d\n",pid);
    else if(pid==0)
    {
        printf("child return value=%d\n",pid);
        printf("child actual process id=%d",getpid());|
    }
    else
        printf("unseccessfull  pid=%d\n",pid);
    return 0;
}
```



Exec() System Call

- Executes a file in an active process

PID = 2502

Active Process

File 1

exec()

PID = 2502

Active Process

File 2

Note: The PID 2502 given above is just for an example purpose.

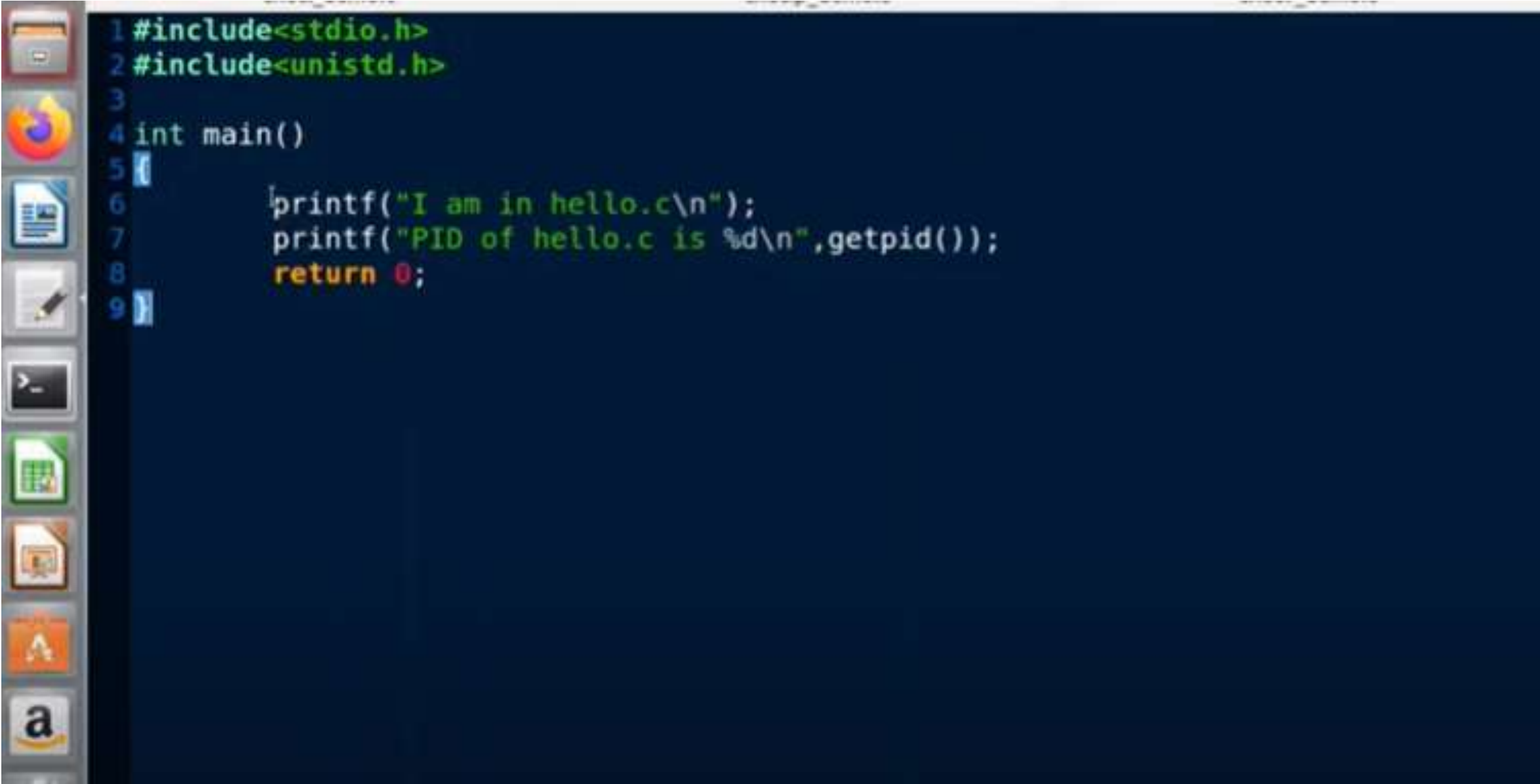
Open ▾

execl_demo.c

execvp_demo.c

execv_demo.c

```
1 #include<stdio.h>
2 #include<unistd.h>
3 int main()
4 {
5     printf("I am in exec_Demo.c\n");
6     printf("PID of exec_demo.c is %d\n",getpid());
7     char *args[] = {"./hello",NULL};
8     execv(args[0],args);
9     printf("coming back to execv_demo.c"); //This will not be executed
10    return 0;
11 }
```



The image shows a screenshot of an Ubuntu desktop environment. On the left side, there is a vertical dock containing several application icons: a file manager, Firefox web browser, LibreOffice Writer, LibreOffice Calc, LibreOffice Impress, and the Amazon logo. The main area of the screen is a dark blue terminal window. Inside the terminal, the following C code is written and numbered from 1 to 9:

```
1 #include<stdio.h>
2 #include<unistd.h>
3
4 int main()
5 {
6     printf("I am in hello.c\n");
7     printf("PID of hello.c is %d\n",getpid());
8     return 0;
9 }
```

buntu@ubuntu: ~/programs

2:06 AM

```
ubuntu@ubuntu:~$ man exec
ubuntu@ubuntu:~$ man exec
ubuntu@ubuntu:~$ cd programs
ubuntu@ubuntu:~/programs$ gcc -o hello hello.c
ubuntu@ubuntu:~/programs$ gcc -o execv_demo execv_demo.c
ubuntu@ubuntu:~/programs$ ./execv_demo
I am in exec Demo.c
PID of exec demo.c is 19026
I am in hello.c
PID of hello.c is 19026
ubuntu@ubuntu:~/programs$
```

Note: The process execution will never return to `execv_demo.c` as the process image has been replaced to `demo.c` file. Hence the last statement in the `execv_demo.c` will not be printed.

<https://stackoverflow.com/questions/26793402/visually-what-happens-to-fork-in-a-for-loop>

```
#include <stdio.h>

void main()
{
    int i;

    for (i=0;i<3;i++)
    {
        fork();

        // This printf statement is for debugging purposes
        // getppid(): gets the parent process-id
        // getpid(): get child process-id

        printf("[%d] [%d] i=%d\n", getppid(), getpid(), i);
    }

    printf("[%d] [%d] hi\n", getppid(), getpid());
}
```

```
[6909][6936] i=0
[6909][6936] i=1
[6936][6938] i=1
[6909][6936] i=2
[6909][6936] hi
[6936][6938] i=2
[6936][6938] hi
[6938][6940] i=2
[6938][6940] hi
[1][6937] i=0
[1][6939] i=2
[1][6939] hi
[1][6937] i=1
[6937][6941] i=1
[1][6937] i=2
[1][6937] hi
[6937][6941] i=2
[6937][6941] hi
[6937][6942] i=2
[6937][6942] hi
[1][6943] i=2
[1][6943] hi
```